

HW5 – Quality Software Presentation

CSC-640

Weeks 1 – 6

Developing and Understanding Quality Software

Week 1 — Foundations of Quality Software

What Is Qualityful Software?

Quality software is software that is:

- ✦ Reliable
- ✦ Maintainable
- ✦ Predictable
- ✦ Easy to evolve

“Quality software is built for change, not for completion.”

A system's *future flexibility* matters just as much as its current correctness.

Managing Complexity

Complexity is the silent killer of quality.

Good engineers **manage** complexity using:

- **Modular design**
- **Clear naming & consistent structure**
- **Abstraction & encapsulation**
- **Documentation & communication**
- **Small feedback loops (testing)**

The goal is not to eliminate complexity —
but to control where it lives.

Core Engineering Principles

KISS – Keep It Simple

- Avoid cleverness.
- Prefer clarity.
- Build only what is necessary.

DRY – Don't Repeat Yourself

- Reduce duplication.
- Prevent duplicated bugs.

Separation of Concerns

- One module → one responsibility.

These principles keep systems predictable and easy to maintain.

“No Surprises” Team Culture

A quality-driven team values:

- Transparency
- Early communication
- Predictable decision-making
- No silent changes

Great software is built by teams with **no surprises**.

Week 1 Reflection

I learned that software quality begins long before coding — in the mindset of:

- Simplicity
- Communication
- Responsibility separation
- Designing with the future in mind

These ideas shaped the way I approached HW4 and the rest of the course.

Week 2 — Software Process

Why Process Matters

A strong process gives teams:

- ✚ Predictability
- ✚ Shared understanding
- ✚ Milestones & direction
- ✚ Controlled risk
- ✚ Repeatable success

Without process, design becomes chaotic and teams lose alignment.

Classic Development Models

Waterfall

- Linear
- Best when requirements are stable

Spiral

- Iterative
- Focuses heavily on **risk analysis**

Agile / Scrum

- Short sprints
- Early customer feedback
- Continuous improvement

HW4 Example — Process in Action

My Student Management REST API followed clear milestones:

- 1 Database schema setup
- 2 Base endpoints
- 3 Add Bearer Token authentication
- 4 Postman + browser testing
- 5 NGINX deployment

Following a structured process kept the project stable, predictable, and on time.

Week 2 Reflection

Process is not “extra work.”

It *reduces* mistakes, improves communication, and prevents major redesigns.

Week 3 — Software Design, OOP, UML

Why OOP Matters

Object-Oriented Programming provides tools to **model real-world systems**:

- **Abstraction** — Focus on essential details
- **Encapsulation** — Protect internal state
- **Inheritance** — Reuse behavior
- **Polymorphism** — Flexible interactions

These reduce complexity in large systems.

◆ APIEC Model (Course Framework)

1. Abstraction
2. Polymorphism
3. Inheritance
4. Encapsulation
5. Composition

APIEC eliminates code smells like `isinstance()` chains and giant switch blocks.

UML — Design Before Coding

UML diagrams help visualize complexity:

- **Use Case Diagrams** — User/system goals
- **Class Diagrams** — Structure & relationships
- **Sequence Diagrams** — Object interactions

This planning prevented confusion during HW4's API design.

HW4 Example — Designing Before Building

- Use cases defined roles (Admins, Students, Courses)
- UML class diagrams modeled relationships
- Controllers → Services → Models built on OOP principles
- Eloquent ORM provided composition & abstraction

Week 3 Reflection

Design is not decoration —
it is the foundation of reliability, scalability, and clarity.

Week 4 — SOLID, Testing, Clean Architecture

SOLID Principles

S – Single Responsibility

One class → one reason to change.

O – Open/Closed

Extend behavior without modifying core code.

L – Liskov Substitution

Subclasses must behave like their parents.

I – Interface Segregation

No “fat” interfaces.

D – Dependency Inversion

Testing = Confidence

Testing ensures:

- Correct behavior
- Safe refactoring
- Early defect detection
- Reliable deployments

Types:

Unit | Integration | System | Regression

Composition vs Inheritance

Too much inheritance → class explosion.

Composition → flexible, testable, adaptable.

HW4 Example — SOLID & Clean Architecture

My REST API demonstrated:

- **SRP** in each controller file
- **DIP** using Models instead of raw SQL
- **OCP** with extendable endpoints
- **Testing** through Postman and automated scripts
- **Separation of concerns** (Routing → Controller → Model)

Week 4 Reflection

SOLID taught me to write code that is easier to maintain, extend, and debug.

Week 5 - High-Level Programming with JavaScript

High-Level Languages

- JavaScript removes low-level details (memory, pointers, manual structures)
- Allows developers to focus on **logic**, not infrastructure
- High-level abstractions lead to **faster development** and **fewer accidental bugs**

Interpretation: High-Level Abstraction

High-level languages like JavaScript simplify development by hiding unnecessary complexity.

This improves productivity and allows engineers to spend their time designing good systems rather than fixing low-level errors.

JavaScript and Web Development

- JavaScript powers modern web interaction
- Native support for JSON → ideal for REST APIs
- Asynchronous tools (Promises, async/await) prevent blocking behavior
- Massive ecosystem (NPM, frameworks, libraries) accelerates development

Interpretation: JavaScript Ecosystem

JavaScript succeeds because it is both accessible and powerful.

Frameworks and libraries reduce repetitive work, while the language itself encourages rapid experimentation.

The ecosystem allows teams to “buy, not build” whenever possible.

Dynamic Typing in JavaScript

- Variables can change type at runtime
- Makes prototyping fast
- BUT increases risk of hidden bugs if code grows large
- Encourages flexible, expressive programming

Interpretation: Dynamic Typing

Dynamic typing boosts creativity early on but creates fragility later.

Without careful testing and documentation, runtime errors can occur in unexpected ways.

This led to the rise of TypeScript in Week 6.

Happy Developers = Better Code

- JavaScript reduces boilerplate
- Faster feedback loops → more enjoyable workflow
- Developers stay motivated when tools feel intuitive

Interpretation: Developer Happiness

When tools minimize frustration, developers write clearer and more maintainable code. JavaScript's simplicity helps teams remain productive and engaged.

HW4 Example: Student Management API

Applying Week 5 Principles:

- **JSON + JavaScript Compatibility**

JavaScript seamlessly processed API responses for testing, debugging, and frontend validation.

- **Asynchronous Requests**

Async/await made interacting with the API responsive and non-blocking.

- **Rapid Prototyping**

Frontend interactions were easy to simulate using browser console or Postman scripts.

HW4 Example: Student Management API

Applying Week 5 Principles:

- **Debugging Visibility**

JavaScript's readable error messages and console tools revealed problems quickly.
Example: malformed JSON or missing fields became immediately visible.

Week 6 - TypeScript & React

Date: December 8th

TypeScript for Reliability

- Adds static typing on top of JavaScript
- Prevents type errors before running the code
- Interfaces and type annotations document system behavior
- Makes refactoring safe and predictable

Interpretation: Static Typing

Strong type systems catch mistakes early.

TypeScript reduces runtime bugs by enforcing structure and consistency during development.

This improves reliability and confidence, especially as applications grow.

TypeScript in Practice

- Enforces shape contracts between components
- IDE autocompletion improves accuracy
- Detects undefined values, wrong arguments, invalid states
- Encourages more disciplined programming

Interpretation: TypeScript Discipline

TypeScript turns JavaScript's flexibility into a strength by adding guardrails. Teams gain clarity, maintainability, and fewer defects — all qualities of long-lasting software.

React — Component-Based Architecture

- UI divided into small, testable components
- One-way data flow reduces complexity
- Virtual DOM improves performance
- Reusable components enable consistency across the system

Interpretation: React Architecture

React provides structure to frontend development.

Instead of chaotic DOM manipulation, engineers create predictable and modular UIs.

This parallels backend frameworks like Laravel in HW4.

Design Patterns in React

- Components
- Props drilling vs context
- Custom hooks
- Container vs presentation components

These patterns improve clarity and collaboration across teams.

Interpretation: UI Patterns

Design patterns provide a shared vocabulary.

By following them, teams avoid reinventing solutions and keep interfaces consistent.

Patterns improve onboarding and accelerate development.

Type Systems + UI Frameworks

When TypeScript and React combine:

- Fewer undefined/null errors
- Safer component communication
- Predictable state transitions
- Clear interfaces for UI and logic
- Fewer runtime surprises

Interpretation: TS + React Synergy

The combination enforces correctness at both data and UI layers.

This results in fewer bugs, smoother collaboration, and scalable system design.

HW4 Example: Student Management API

Applying Week 6 Principles:

- **Architecture vs Component Design**

The backend had controllers, models, and middleware.

A React frontend would mirror this with components, hooks, and services.

- **Type Safety**

Student objects (id, name, email, etc.) would be represented with TypeScript interfaces, reducing frontend errors.

HW4 Example: Student Management API

Applying Week 6 Principles:

- **React UI Flow**

Fetch students → Display list → Show details → Update forms

Component-based architecture enables clarity and modularity.

- **TypeScript Contracts**

Ensures API responses match expected structures, catching mismatches early.